Applied Type Theory

Лекция 3: просто типизированное λ-исчисление

Воронов Михаил Сергеевич

ВМК МГУ · Осень 2026

Задача лекции: проверить программу до запуска

Есть две функции и аргумент

$$f : \beta \to \gamma \qquad g : \alpha \to \beta \qquad x : \alpha$$

Какое выражение имеет тип γ, если α, β, γ — различные атомарные типы?

$$f\,(g\,x) \qquad\qquad g\,(f\,x)$$

Можно ли проверить ответ, **не вычисляя** ни f, ни g?

Построим формальные правила, затем докажем, что вычисление **сохраняет тип** программы.

План лекции

1 **Мотивация и роль систем типов**

2 Простые типы, правила и алгоритмы

3 Метатеория: сохранение, прогресс, нормализация

Вычисления в нетипизированном исчислении

Общая рекурсия

$$YF =_\beta F(YF)$$

Неподвижная точка позволяет выразить рекурсивную программу. Её завершение требует отдельного обоснования.

Бесконечное вычисление

$$\Omega := (\lambda x.x\,x)(\lambda x.x\,x) \to_\beta \Omega \to_\beta \cdots$$

Простые типы выделяют класс термов, у которых **все последовательности** β-редукций конечны.

Логическая непротиворечивость относится к выбранной логике и её интерпретации. Нетерминация сама по себе не является логическим противоречием.

Что описывает тип

Значения и допустимые операции

Тип классифицирует выражения и определяет, **как их можно использовать** в программе.

$$g : \alpha \to \beta$$

Функцию g можно применить к аргументу типа α. Результат имеет тип β.

Разные уровни описания

- **Представление**: как данные хранятся в памяти.
- **Семантика**: какие значения обозначает тип.
- **Правила типизации**: какие выражения получают этот тип.

Теория типов: логика и программирование

Логические основания

Типовые ограничения возникли при изучении логических парадоксов. В 1940 году Чёрч связал простую теорию типов с λ-абстракцией.

Автоматический вывод типов

Работы Милнера (1978) и Дамаса–Милнера (1982) связали полиморфизм ML с алгоритмами вывода и главными типовыми схемами.

Доказательства как программы

Соответствие Карри–Ховарда связывает типы с формулами. Зависимые типы позволяют формулировать утверждения о значениях внутри типов.

В этой лекции рассматриваем **простые стрелочные типы**. Полиморфизм и зависимые типы появятся далее.

Гарантии системы типов

Система типов проверяет программу по её синтаксису и исключает определённые виды ошибочного поведения.

Статическая проверка консервативна

Некоторые вычисляющиеся программы могут не получить тип. **Отказ типизации не означает**, что выполнение обязательно завершится ошибкой.

Свойства зависят от языка

- **Безопасность типов**: вычисление не попадает в определённый семантикой тупик.
- **Нормализация**: вычисление обязательно завершается.

Для чистого STLC доказываются оба свойства. При добавлении общей рекурсии нормализация теряется.

План лекции

1 Мотивация и роль систем типов

2 Простые типы, правила и алгоритмы

3 Метатеория: сохранение, прогресс, нормализация

Простые типы и обозначения

Индуктивное определение

$$\sigma, \tau ::= \alpha \mid \sigma \to \tau$$

$\alpha, \beta, \gamma, \ldots$ обозначают **атомарные типы**, а $\sigma, \tau, \rho, \ldots$ — **произвольные простые типы**.

Конечное дерево типа

$$(\alpha \to \beta) \to \alpha \to \beta$$

В STLC нет рекурсивных уравнений типов и квантора ∀.

В алгоритме вывода введём **отдельные неизвестные** $X, Y, Z, \ldots$. Подстановка в схему типа выполняется на метауровне.

Стрелочные типы и порядок скобок

Стрелка ассоциативна вправо

$$\alpha \to \beta \to \gamma \equiv \alpha \to (\beta \to \gamma)$$

Функция принимает α и возвращает функцию типа $\beta \to \gamma$. Так записывается каррирование.

$$(\alpha \to \beta) \to \gamma$$

Здесь аргументом служит функция типа $\alpha \to \beta$. **Скобки меняют тип**.

Применение ассоциативно влево

$$f\ x\ y \equiv (f\ x)\ y \qquad \lambda f\ x.M \equiv \lambda f.\lambda x.M$$

Термы в стиле Карри и в стиле Чёрча

Стиль Карри

$$M, N ::= x \mid M\,N \mid \lambda x.M$$

Аннотаций нет. Для терма ищем вывод типизации.

Стиль Чёрча

$$M, N ::= x \mid M\,N \mid \lambda x : \sigma.M$$

Тип каждого параметра записан в абстракции.

Это два варианта синтаксиса. Наличие аннотаций ещё не гарантирует, что терм корректно типизирован.

Контекст типизации

Контекст Γ — конечное отображение переменных в типы. **Одному имени соответствует один тип**.

$$\Gamma \coloneqq f : \beta \to \gamma,\ g : \alpha \to \beta,\ x : \alpha$$

$$\mathrm{dom}(\Gamma) = \{f, g, x\} \qquad \Gamma(x) = \alpha$$

Расширение и сужение

$\Gamma, y : \sigma$ добавляет новое имя $y \notin \mathrm{dom}(\Gamma)$. Порядок объявлений здесь несущественен.

$\Gamma \restriction V$ оставляет только объявления переменных из множества V.

При конфликте с именем связанной переменной сначала выполняем α-переименование.

Суждение о типизации

$$\Gamma \vdash M : \tau$$

При предположениях из Γ терм M имеет тип τ.

Суждение верно, когда существует **конечное дерево вывода** по правилам типизации.

$$f : \alpha \to \beta,\ x : \alpha \vdash f\ x : \beta$$

$\vdash M : \tau$ означает **пустой контекст**. Такой типизуемый терм не имеет свободных переменных.

Правила для переменной и применения

Переменная: Var

$$\frac{x : \sigma \in \Gamma}{\Gamma \vdash x : \sigma}\ \text{Var}$$

Применение: App, или $\to E$

$$\frac{\Gamma \vdash M : \sigma \to \tau \qquad \Gamma \vdash N : \sigma}{\Gamma \vdash M\,N : \tau}\ \text{App}$$

Тип аргумента **должен совпадать** с входным типом функции.

Над чертой записаны посылки, под чертой — заключение. Оба правила одинаковы для стилей Карри и Чёрча.

Правило для абстракции

Стиль Карри

$$\frac{\Gamma, x : \sigma \vdash M : \tau}{\Gamma \vdash \lambda x.M : \sigma \to \tau}\ \text{Abs}$$

Стиль Чёрча

$$\frac{\Gamma, x : \sigma \vdash M : \tau}{\Gamma \vdash \lambda x : \sigma.M : \sigma \to \tau}\ \text{Abs}$$

В стиле Чёрча тип σ берём **из аннотации**. В стиле Карри он входит в искомый вывод.

Условие: $x \notin \mathrm{dom}(\Gamma)$. После абстракции предположение о x **исчезает из внешнего контекста**.

Структурные свойства контекста

Пусть $\Gamma \vdash M : \tau$. Тогда

Все свободные переменные объявлены

$$\mathrm{FV}(M) \subseteq \mathrm{dom}(\Gamma)$$

Ослабление

$$\Gamma \subseteq \Delta \quad \Longrightarrow \quad \Delta \vdash M : \tau$$

Дополнительные объявления сохраняют старые типы переменных.

Удаление неиспользуемых объявлений

$$\Gamma \restriction \mathrm{FV}(M) \vdash M : \tau$$

Все три свойства доказываются **индукцией по дереву типизации**. Они понадобятся в лемме о подстановке.

Композиция: тип тела функции

Пусть $\Gamma = f : \beta \to \gamma,\ g : \alpha \to \beta,\ x : \alpha$.

$$\Gamma \vdash g\,x : \beta$$

$$\Gamma \vdash f\,(g\,x) : \gamma$$

$$\dfrac{\dfrac{f : \beta \to \gamma \in \Gamma}{\Gamma \vdash f : \beta \to \gamma}\ \text{Var} \qquad \dfrac{\Gamma \vdash g : \alpha \to \beta \qquad \Gamma \vdash x : \alpha}{\Gamma \vdash g\,x : \beta}\ \text{App}}{\Gamma \vdash f\,(g\,x) : \gamma}\ \text{App}$$

Сначала результат $g\,x$ получает тип β. **Именно такой аргумент** нужен функции f.

Композиция: снятие предположений

Три применения правила Abs

$$f : \beta \to \gamma, g : \alpha \to \beta, x : \alpha \vdash f(gx) : \gamma$$

$$f : \beta \to \gamma, g : \alpha \to \beta \vdash \lambda x.f(gx) : \alpha \to \gamma$$

$$f : \beta \to \gamma \vdash \lambda g\, x.f(gx) : (\alpha \to \beta) \to \alpha \to \gamma$$

$$\vdash \lambda f\, g\, x.f(gx) : (\beta \to \gamma) \to (\alpha \to \beta) \to \alpha \to \gamma$$

Параметры абстрагируются **справа налево**: x, затем g, затем f. Итоговый терм **замкнут**.

Композиция во флаговой нотации

1	$f : \beta \to \gamma$	допущение
2	$g : \alpha \to \beta$	допущение
3	$x : \alpha$	допущение
4	$g\,x : \beta$	App, строки 2 и 3
5	$f(gx) : \gamma$	App, строки 1 и 4
6	$\lambda x.f(gx) : \alpha \to \gamma$	Abs, закрываем 3
7	$\lambda g\,x.f(gx) : (\alpha \to \beta) \to \alpha \to \gamma$	Abs, закрываем 2
8	$\lambda f\,g\,x.f(gx) : (\beta \to \gamma) \to (\alpha \to \beta) \to \alpha \to \gamma$	Abs, закрываем 1

Активные допущения образуют контекст текущей строки. **Закрытое допущение** больше нельзя использовать снаружи абстракции.

Повторное использование переменной

$\lambda x\, y.x\, (y\, x)$

Оба вхождения переменной x должны иметь **один тип**. Выберем $x : \beta \to \delta$.

$y : (\beta \to \delta) \to \beta \qquad y\, x : \beta$

$x\, (y\, x) : \delta$

$\vdash \lambda x\, y.x(yx) : (\beta \to \delta) \to ((\beta \to \delta) \to \beta) \to \delta$

Вхождение x как аргумента функции y допустимо. Конфликта с его ролью функции во внешнем применении нет.

Аннотации и стирание типов

Тот же пример в стиле Чёрча

$$\lambda x : \beta \to \delta.\lambda y : (\beta \to \delta) \to \beta.x(yx)$$

$$: (\beta \to \delta) \to ((\beta \to \delta) \to \beta) \to \delta$$

Стирание аннотаций

$$|\lambda x : \sigma.M| \coloneqq \lambda x.|M|$$

Стирание сохраняет переменные и применения. Из типизируемого терма Чёрча получаем типизируемый терм Карри.

Обратно, по дереву типизации терма Карри можно **восстановить аннотации** параметров. Они зависят от выбранного вывода.

Основные задачи типизации

Задача	Дано	Нужно найти
Вывод типа	Γ, M	**тип** τ, для которого $\Gamma \vdash M : \tau$
Проверка типа	Γ, M, τ	**ответ**, выводимо ли $\Gamma \vdash M : \tau$
Обитаемость	Γ, τ	**терм** M, для которого $\Gamma \vdash M : \tau$

При выводе типа для открытого терма можно также искать подходящий контекст. Тогда результатом служит пара «контекст и тип».

Вывод типа через ограничения

Пример: применение функции к аргументу

$$M \coloneqq \lambda f\ x.f\ x$$

Введём неизвестные типы: $f : X$, $x : Y$, результат $f\ x : Z$.

$$X = Y \to Z$$

$$\vdash \lambda f\ x.f\ x : (Y \to Z) \to Y \to Z$$

Неизвестные Y и Z остались свободными. Можно подставить вместо них любые простые типы.

Все вхождения одной связанной переменной используют **одну и ту же неизвестную** типа.

Композиция: решение уравнений на типы

$\lambda f\ g\ x.f(gx)$

Пусть $f : F$, $g : G$, $x : X$. Обозначим типы $gx : Y$ и $f(gx) : Z$.

$G = X \to Y \qquad F = Y \to Z$

$F \to G \to X \to Z \quad \rightsquigarrow \quad (Y \to Z) \to (X \to Y) \to X \to Z$

Переименуем X, Y, Z в α, β, γ.

$(\beta \to \gamma) \to (\alpha \to \beta) \to \alpha \to \gamma$

Получили **тот же тип**, что и при построении дерева вывода.

Унификация простых типов

Решаем уравнения над конечными деревьями

$$\sigma_1 \to \sigma_2 = \tau_1 \to \tau_2 \quad \rightsquigarrow \quad \{\sigma_1 = \tau_1,\ \sigma_2 = \tau_2\}$$

Одинаковые типы дают тождество. Различные фиксированные атомы несовместимы, как и атом со стрелочным типом.

$$X = \tau \qquad X \notin \mathrm{FTV}(\tau) \qquad \rightsquigarrow \qquad [X := \tau]$$

Подстановку применяем **ко всем оставшимся уравнениям** и уже найденному результату.

Проверка вхождения: occurs-check

Сначала удаляем $X = X$. Если X входит в другой тип τ, уравнение $X = \tau$ **не имеет конечного решения**.

Почему $x\,x$ не получает простой тип

Предположим, что $\Gamma \vdash x\,x : \tau$, а единственный тип переменной x в контексте равен T.

$$T = T \to \tau$$

Размер конечного типа

$$|\alpha| = 0 \qquad |\sigma \to \rho| = 1 + |\sigma| + |\rho|$$

$$|T| = 1 + |T| + |\tau|$$

Получили противоречие. Так occurs-check обнаруживает несовместимые требования **к одной переменной**.

Почему $I\,I$ всё же типизируется

Пусть I — сокращение для $\lambda x.x$ в стиле Карри. Два вхождения сокращения **раскрываются независимо**.

$$I\,I \equiv (\lambda u.u)(\lambda v.v)$$

$$\lambda u.u : (\alpha \to \alpha) \to (\alpha \to \alpha) \qquad \lambda v.v : \alpha \to \alpha$$

$$\vdash I\,I : \alpha \to \alpha \qquad \underline{I\,I} \to_\beta I$$

У переменной i в контексте **один фиксированный тип**. Поэтому $i\,i$ не типизируется.

В раскрытом $I\,I$ используются два отдельных вывода. Полиморфное связывание let для этого не требуется.

Главный тип в стиле Карри

Наиболее общий результат вывода

Для каждого замкнутого типизуемого терма M существует шаблон типа π, из которого подстановкой получаются **все типы** M.

$$\pi_I = X \to X$$

$$\pi_I[X := \alpha] = \alpha \to \alpha$$

$$\pi_I[X := \alpha \to \beta] = (\alpha \to \beta) \to (\alpha \to \beta)$$

Алгоритм унификации даёт наиболее общий результат и не фиксирует типы раньше, чем этого требуют ограничения.

Главный тип — **схема на метауровне**. Квантор ∀ в синтаксисе STLC не появляется.

Проверка типов в стиле Чёрча

Рекурсивный алгоритм синтеза типа

$\text{type}_\Gamma(x) = \Gamma(x)$

$\text{type}_\Gamma(\lambda x : \sigma.M) = \sigma \to \text{type}_{\Gamma, x:\sigma}(M)$

Для MN вычисляем типы обеих частей. Если они имеют вид $\sigma \to \tau$ и σ, **возвращаем** τ. Иначе сообщаем об ошибке.

Проверка заданного типа

$\Gamma \vdash M : \tau \quad \Longleftrightarrow \quad \text{type}_\Gamma(M)$ определён и равен τ

Алгоритм завершается: рекурсивные вызовы обрабатывают **собственные подтермы**. Равенство типов проверяется структурно.

Двунаправленная типизация

$\Gamma \vdash M \Rightarrow \tau$ синтез $\quad$ $\Gamma \vdash M \Leftarrow \tau$ проверка

В этом алгоритмическом варианте используем термы Карри и полную аннотацию выражения $(M : \tau)$.

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x \Rightarrow \tau} \qquad \frac{\Gamma \vdash M \Rightarrow \sigma \to \tau \qquad \Gamma \vdash N \Leftarrow \sigma}{\Gamma \vdash M\,N \Rightarrow \tau}$$

$$\frac{\Gamma, x : \sigma \vdash M \Leftarrow \tau}{\Gamma \vdash \lambda x.M \Leftarrow \sigma \to \tau}$$

Применение получает входной тип из функции. При проверке абстракции тип параметра приходит **из ожидаемого стрелочного типа**.

Переход между синтезом и проверкой

$$\frac{\Gamma \vdash M \Rightarrow \sigma \qquad \sigma \equiv \tau}{\Gamma \vdash M \Leftarrow \tau} \qquad \frac{\Gamma \vdash M \Leftarrow \tau}{\Gamma \vdash (M : \tau) \Rightarrow \tau}$$

Пример с ожидаемым типом аргумента

$h : (\alpha \to \alpha) \to \beta \vdash h\,(\lambda x.x) \Rightarrow \beta$

$x : \alpha \vdash x \Rightarrow \alpha \quad \Longrightarrow \quad x : \alpha \vdash x \Leftarrow \alpha$

$\vdash \lambda x.x \Leftarrow \alpha \to \alpha$

Полная аннотация $(\lambda x.x : \alpha \to \alpha)$ позволяет синтезировать тип абстракции.

Обитаемость типа и поиск программы

Терм неизвестен, тип задан

Найти M такое, что $\Gamma \vdash M : \sigma \to \tau$

Можно начать с λx и искать тело типа τ в контексте $\Gamma, x : \sigma$.

$\vdash \lambda f\ x.f\ x : (\alpha \to \beta) \to \alpha \to \beta$

Соответствие Карри–Ховарда

Стрелочные типы читаются как **импликации**, а термы с выводом типизации — как **доказательства**.

Обитаемость чистых простых типов разрешима, но в общем случае PSPACE-полна. Поиск программы может быть значительно **труднее её проверки**.

Пример необитаемого типа

$$(\alpha \to \beta \to \alpha) \to \beta \to \alpha$$

Пусть α и β — **различные атомы**. После введения параметров имеем $f : \alpha \to \beta \to \alpha$ и $y : \beta$, а нужен результат типа α.

Контроценка для логической формулы

$\alpha =$ ложь $\qquad \beta =$ истина

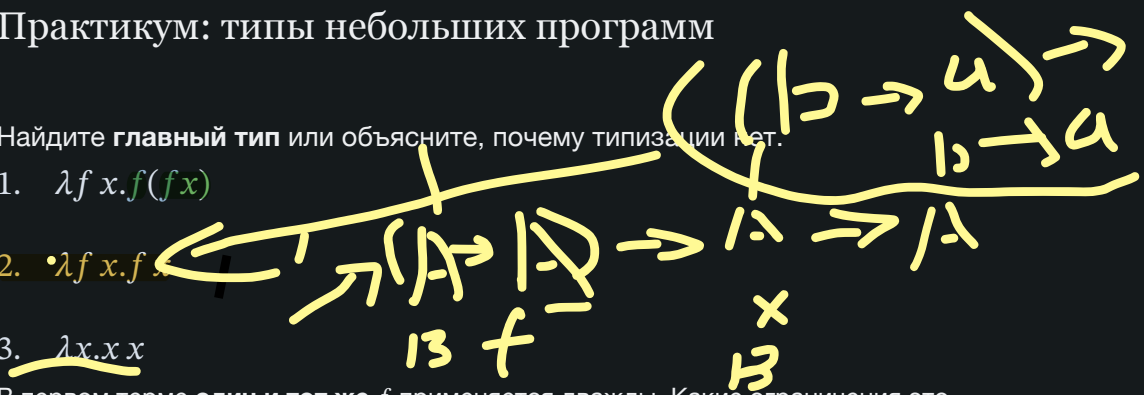
$\alpha \to \beta \to \alpha =$ истина $\qquad (\alpha \to \beta \to \alpha) \to \beta \to \alpha =$ ложь

Правила введения и устранения импликации сохраняют истинность. Поэтому у этой формулы **нет доказательства**, а у типа — замкнутого обитателя.

Практикум: типы небольших программ

Найдите **главный тип** или объясните, почему типизации нет.

1. $\lambda f\ x.f(f\,x)$
2. $\lambda f\ x.f\ x$
3. $\lambda x.x\ x$



В первом терме **один и тот же** f применяется дважды. Какие ограничения это накладывает на его вход и выход?

Дерево вывода для двукратного применения

Обозначим $\Delta = f : \alpha \to \alpha,\ x : \alpha$.

$$\dfrac{\Delta \vdash f : \alpha \to \alpha \qquad \dfrac{\Delta \vdash f : \alpha \to \alpha \qquad \Delta \vdash x : \alpha}{\Delta \vdash f\,x : \alpha}\ \text{App}}{\Delta \vdash f(f\,x) : \alpha}\ \text{App}$$

$$f : \alpha \to \alpha \vdash \lambda x.f(f\,x) : \alpha \to \alpha$$

$$\vdash \lambda f\ x.f(f\,x) : (\alpha \to \alpha) \to \alpha \to \alpha$$

Внутреннее применение возвращает α. Внешнее применение принимает **тот же тип**.

Аннотации и способ задания языка

Карри и Чёрч: где записаны типы

В стиле Карри терм не содержит аннотаций параметров. В стиле Чёрча они входят в синтаксис.

Внешнее и внутреннее представление типизации

При **внешнем подходе** сначала задают сырые термы, затем отношение $\Gamma \vdash M : \tau$. При **внутреннем подходе** корректные типы входят в само представление термов.

Это связанные, но разные различия. Аннотированный синтаксис можно снабдить внешним отношением типизации.

Наличие вывода типов или отсутствие аннотаций в исходном коде само по себе не определяет безопасность языка.

План лекции

1 Мотивация и роль систем типов

2 Простые типы, правила и алгоритмы

3 Метатеория: сохранение, прогресс, нормализация

Инверсия правил типизации

Форма терма определяет последнее правило

$$\Gamma \vdash x : \tau \quad \Longrightarrow \quad x : \tau \in \Gamma$$

$$\Gamma \vdash M\,N : \tau \quad \Longrightarrow \quad \exists \sigma.\, \Gamma \vdash M : \sigma \to \tau \ \wedge\ \Gamma \vdash N : \sigma$$

$$\Gamma \vdash \lambda x.M : \rho \quad \Longrightarrow \quad \exists \sigma, \tau.\, \rho \equiv \sigma \to \tau \ \wedge\ \Gamma, x : \sigma \vdash M : \tau$$

В стиле Чёрча σ в последней строке фиксируется аннотацией $\lambda x : \sigma.M$.

Доказательство: разбираем **последнее правило имеющегося вывода**. Других правил, способных дать терм такой формы, нет.

Типы подтермов и связанные переменные

Если $\Gamma \vdash M : \tau$, то каждое вхождение подтерма имеет тип **в подходящем контексте**.

$\vdash \lambda x : \alpha . x : \alpha \to \alpha$

$x : \alpha \vdash x : \alpha$ но $\not\vdash x : \alpha$

При переходе под λ **контекст расширяется** типом параметра. Поэтому подтерм может не типизироваться в исходном Γ.

Лемма следует из инверсии: для применения получаем обе посылки, для абстракции — типизацию тела с дополнительным объявлением.

Единственность типа в стиле Чёрча

При фиксированном контексте и аннотированном терме

$$\Gamma \vdash M : \sigma \quad \wedge \quad \Gamma \vdash M : \tau \quad \Longrightarrow \quad \sigma \equiv \tau$$

Индукция по структуре терма

- Переменная: её единственный тип записан в Γ.
- Применение: по индукции тип функции единственен, значит единственен и тип результата.
- Абстракция: аннотация фиксирует тип параметра, а индукция — тип тела.

В стиле Карри $\lambda x.x$ имеет разные типы. Их объединяет **главный шаблон** $X \to X$.

Подстановка в типах и контекстах

Подстановка S заменяет параметры схемы простыми типами и **сохраняет стрелочную структуру**.

$$S(\sigma \to \tau) = S(\sigma) \to S(\tau)$$

$$S(\Gamma) = \{x : S(\Gamma(x)) \mid x \in \mathrm{dom}(\Gamma)\}$$



Пример

$$S = [\alpha := \beta \to \beta]$$

$$S(\alpha \to \alpha) = (\beta \to \beta) \to (\beta \to \beta)$$

Это операция над типами и выводами. Программа не делает при этом β-шаг.

Подстановка типов в двух стилях

Стиль Карри: терм остаётся тем же

$$\Gamma \vdash M : \tau \quad \Longrightarrow \quad S(\Gamma) \vdash M : S(\tau)$$

Стиль Чёрча: меняются аннотации

$$\Gamma \vdash M : \tau \quad \Longrightarrow \quad S(\Gamma) \vdash S(M) : S(\tau)$$

$$S(\lambda x : \sigma.M) = \lambda x : S(\sigma).S(M)$$

$$S(\lambda x : \alpha.x) \equiv \lambda x : \beta \to \beta.x \qquad S = [\alpha := \beta \to \beta]$$

Доказательство: индукция по дереву типизации с применением S ко всем типам в каждом правиле.

Лемма о подстановке терма

Аргумент заменяет параметр того же типа

$$\frac{\Gamma, x : \sigma \vdash M : \tau \qquad \Gamma \vdash N : \sigma}{\Gamma \vdash M[x := N] : \tau}$$

Условие: $x \notin \mathrm{dom}(\Gamma)$. Подстановка выполняется **без захвата** свободных переменных.

Пример

$$f : \alpha \to \beta, x : \alpha \vdash f\ x : \beta$$

Если $\Gamma = f : \alpha \to \beta, y : \alpha$, то подстановка $x := y$ даёт $\Gamma \vdash f\ y : \beta$.

Лемма связывает правило применения с вычислительным правилом β-редукции.

Подстановка: ключевые случаи доказательства

Индукция по выводу $\Gamma, x : \sigma \vdash M : \tau$

- $M \equiv x$: результатом становится N, нужный тип уже дан во второй предпосылке.
- $M \equiv y$, где $y \neq x$: объявление y остаётся в Γ.
- $M \equiv M_1 M_2$: применяем индукцию к двум посылкам, затем восстанавливаем App.

Абстракция $M \equiv \lambda y.P$

Выбираем $y \neq x$ **свежим** для Γ, N. **Ослабление** даёт $\Gamma, y : \rho \vdash N : \sigma$.

$$\Gamma, y : \rho, x : \sigma \vdash P : \zeta \quad \Longrightarrow \quad \Gamma, y : \rho \vdash P[x := N] : \zeta$$

По Abs получаем $\Gamma \vdash \lambda y.P[x := N] : \rho \to \zeta$.

Сохранение типа при β-редукции

$$\Gamma \vdash M : \tau \quad \wedge \quad M \to_\beta N \quad \Longrightarrow \quad \Gamma \vdash N : \tau$$

Основной случай

$$\underline{(\lambda x : \sigma . P)\, Q} \to_\beta P[x := Q]$$

Инверсия App и Abs восстанавливает две посылки.

$$\Gamma, x : \sigma \vdash P : \tau \qquad \Gamma \vdash Q : \sigma$$

Лемма о подстановке даёт тип результата.

$$\Gamma \vdash P[x := Q] : \tau$$

При редукции в подтерме применяем индукцию и восстанавливаем правило для всего терма. Это включает редукцию под λ.

Тип как инвариант вычисления

Пусть $\Gamma = g : \alpha \to \beta, x : \alpha$. Тогда

$\Gamma \vdash (\lambda f : \alpha \to \beta . \lambda y : \alpha . f\ y)\ g\ x : \beta$

$\underline{(\lambda f : \alpha \to \beta . \lambda y : \alpha . f\ y)\ g}\ x \to_\beta \underline{(\lambda y : \alpha . g\ y)\ x} \to_\beta g\ x$

$\Gamma \vdash g\ x : \beta$

Повторяя теорему **для каждого шага**, получаем сохранение типа вдоль любой конечной цепочки редукций.

$$\Gamma \vdash M : \tau \quad \wedge \quad M \to_\beta^* N \quad \Longrightarrow \quad \Gamma \vdash N : \tau$$

Редукция может сделать терм типизируемым

Обратное к сохранению утверждение неверно

Возьмём $I \coloneqq \lambda z.z$ в стиле Карри.

$$\underline{(\lambda x.x\,x)\,I} \to_\beta \underline{I\,I} \to_\beta I$$

- **Исходный терм не типизируется**: внутри него есть $x\,x$.
- Уже **после первого шага** $I\,I$ имеет тип $\alpha \to \alpha$.
- Вычисление завершается, хотя исходная программа не получила простой тип.

Таким образом, типизируемость сохраняется по направлению редукции, но не по направлению произвольного β-расширения.

Нетипизируемость Ω и Y

$\Omega \coloneqq (\lambda x.x\ x)(\lambda x.x\ x)$

$Y \coloneqq \lambda f.(\lambda x.f(x\ x))(\lambda x.f(x\ x))$

Оба терма содержат подтерм $x\ x$ под связыванием x. Этот подтерм не получает простой тип ни в каком контексте.

По лемме о типизируемости подтермов типизации всего Ω и всего Y не существует.

Мы исключили два конкретных терма. Завершение всех типизируемых программ требует **отдельной теоремы**.

Сильная нормализация STLC

Теорема для чистого просто типизированного исчисления

$$\Gamma \vdash M : \tau \quad \Longrightarrow \quad M \in \mathrm{SN}$$

Не существует бесконечной последовательности β-редукций, начинающейся с M. Это относится и к **открытым термам**.

- Любая максимальная последовательность редукций заканчивается в нормальной форме.
- По теореме Чёрча–Россера нормальная форма единственна с точностью до α.

Нормализация не следует из одной лишь безопасности типов. В доказательстве нужно учесть поведение функций на всех подходящих аргументах.

Идея доказательства нормализации

Редуцируемость определяется по структуре типа

$$\mathcal{R}_\alpha := \mathrm{SN}$$

$$\mathcal{R}_{\sigma \to \tau} := \{M \mid \forall N \in \mathcal{R}_\sigma.\ M\,N \in \mathcal{R}_\tau\}$$

Сначала доказывают, что $\mathcal{R}_\tau \subseteq \mathrm{SN}$, и что переменные принадлежат нужным классам редуцируемости.

Основная лемма

Если заменить свободные переменные типизируемого терма **редуцируемыми термами соответствующих типов**, результат тоже редуцируем.

Подставляем каждую переменную саму в себя. Получаем $M \in \mathcal{R}_\tau$, откуда $M \in \mathrm{SN}$.

Расширение с булевыми значениями

Для примеров безопасности добавим базовый тип Bool, константы и условное выражение.

$$M, N ::= \cdots \mid \mathsf{true} \mid \mathsf{false} \mid \mathsf{if}\, M\, \mathsf{then}\, N\, \mathsf{else}\, P$$

$$\Gamma \vdash \mathsf{true} : \mathsf{Bool} \qquad \Gamma \vdash \mathsf{false} : \mathsf{Bool}$$

$$\frac{\Gamma \vdash L : \mathsf{Bool} \qquad \Gamma \vdash M : \tau \qquad \Gamma \vdash N : \tau}{\Gamma \vdash \mathsf{if}\, L\, \mathsf{then}\, M\, \mathsf{else}\, N : \tau}$$

Условие имеет тип Bool, а обе ветви — **один и тот же тип** τ.

Здесь true и false — встроенные константы. Их вычислительные правила зададим отдельно.

Стратегия вычислений для теоремы прогресса

Call-by-value: значения и контексты

$V ::= \lambda x : \sigma.M \mid \text{true} \mid \text{false}$

$E ::= [\,] \mid E\,M \mid V\,E \mid \text{if } E \text{ then } M \text{ else } N$

$$\frac{}{(\lambda x : \sigma.M)\,V \to_{\text{v}} M[x := V]}$$

$$\text{if true then } M \text{ else } N \to_{\text{v}} M \qquad \text{if false then } M \text{ else } N \to_{\text{v}} N$$

Любой такой шаг разрешён внутри E. **Под λ не заходим**, невыбранную ветвь не вычисляем.

Канонические формы и прогресс

Форма замкнутого типизируемого значения

$$\vdash V : \sigma \to \tau \quad \Longrightarrow \quad V \equiv \lambda x : \sigma . M$$

$$\vdash V : \mathsf{Bool} \quad \Longrightarrow \quad V \in \{\mathsf{true}, \mathsf{false}\}$$

Теорема прогресса

$$\vdash M : \tau \quad \Longrightarrow \quad M \text{ — значение} \quad \vee \quad \exists N.\, M \to_{\mathrm{v}} N$$

Замкнутая типизируемая программа либо уже вычислена до значения, **либо может сделать следующий шаг**.

Замкнутость существенна: $x : \mathsf{Bool} \vdash x : \mathsf{Bool}$, но свободная переменная не является значением и не делает шага.

Почему типизируемая программа делает шаг

Индукция по выводу типизации замкнутого терма

- Переменная невозможна в пустом контексте. Абстракции и булевы константы уже являются значениями.
- В применении сначала делает шаг функция, затем аргумент. Если оба — значения, **каноническая форма** функции даёт λ-абстракцию и β-шаг.
- В условном выражении вычисляется условие. Если оно уже значение типа Bool, можно выбрать одну из ветвей.

Значение не обязано быть полной нормальной формой

$$\lambda x : \alpha.\underline{(\lambda y : \alpha.y)\, x}$$

Для CBV это **готовое значение**. Полная β-редукция могла бы продолжиться под абстракцией.

Безопасность типов: отсутствие тупиков

Терм находится в тупике, если он **не является значением** и **не имеет следующего шага**.

true false тупик, но терм не типизируется

Прогресс и сохранение вместе

$$\vdash M : \tau \quad \wedge \quad M \to_{\mathrm{v}}^{*} N \quad \Longrightarrow \quad \vdash N : \tau$$

Сохранение оставляет результат типизируемым и замкнутым. Прогресс даёт значение или следующий шаг. Поэтому N не находится в тупике.

Этот аргумент применим к любому конечному префиксу вычисления. Сам по себе он **не исключает бесконечную последовательность** шагов.

Безопасность без гарантии завершения

Добавим отдельный примитив рекурсии

$$\frac{\Gamma \vdash F : \tau \to \tau}{\Gamma \vdash \mathsf{fix}\, F : \tau}$$

$$\mathsf{fix}(\lambda x : \tau.M) \to_{\mathrm{v}} M[x := \mathsf{fix}(\lambda x : \tau.M)]$$

Сначала вычисляем аргумент fix до абстракции. Затем применяем показанное правило раскрытия.

Типизируемая бесконечная программа

$$\vdash \mathsf{fix}(\lambda x : \mathsf{Bool}.x) : \mathsf{Bool}$$

$$\mathsf{fix}(\lambda x : \mathsf{Bool}.x) \to_{\mathrm{v}} \mathsf{fix}(\lambda x : \mathsf{Bool}.x) \to_{\mathrm{v}} \cdots$$

Для этого расширения **сохраняются прогресс и типы**, но **сильной нормализации уже нет**.

Общая рекурсия и логическая интерпретация

Примитив fix добавляет новое правило. Он не делает нетипизируемый комбинатор Y типизируемым в прежнем STLC.

Цена для интерпретации типов как утверждений

$$\vdash \mathsf{fix}(\lambda x : \tau.x) : \tau$$

В расширении с общей рекурсией можно построить **расходящийся терм любого типа** τ.

Чтобы понимать термы как доказательства, нужно дополнительно контролировать **допустимую рекурсию и завершение**.

Практический язык может разрешать общую рекурсию и сохранять безопасность типов. Полнота по Тьюрингу сама по себе не противоречит безопасности.

Тот же пример в OCaml

Определение композиции

```
let compose f g x = f (g x)
```

Тип, выведенный компилятором

```
('a -> 'b) -> ('c -> 'a) -> 'c -> 'b
```

$$(\beta \to \gamma) \to (\alpha \to \beta) \to \alpha \to \gamma$$

Имена переменных типа могут различаться, но **структура типа совпадает** с нашим выводом.

OCaml дополнительно обобщает допустимые переменные типа при let-связывании. Это следующий шаг после мономорфного STLC.

Синтез программы с проверкой типов

Задача для генератора программы

$$f : \beta \to \gamma,\ g : \alpha \to \beta,\ x : \alpha \vdash \boxed{?} : \gamma$$

Нужно построить терм, удовлетворяющий заданному суждению. Можно использовать модель для **предложения кандидатов**, а правила типов — **для их проверки**.

$$g\,x : \beta \qquad \Longrightarrow \qquad f(gx) : \gamma$$

Типы также задают подзадачи: чтобы применить f, нужен аргумент типа β. Его можно получить через g.

Проверка типа гарантирует соблюдение этого интерфейса. Более точные свойства поведения требуют дополнительных спецификаций.

Как расширяется простое исчисление

Данные и способы их обработки

Произведения, суммы и алгебраические типы данных добавляют структуру значений и правила сопоставления с образцом.

Полиморфизм

Hindley–Milner и System F позволяют выразить повторное использование программы при разных типах.

Более точные ограничения

Зависимые типы описывают **свойства значений**. Линейные и аффинные дисциплины ограничивают **использование ресурсов**.

При каждом расширении заново проверяем, какие леммы и гарантии сохранились. Следующая тема курса — алгебраические типы данных.

Что дают правила простой типизации

Построение и проверка программ

Var, App и Abs задают конечное дерево вывода. Аннотации позволяют проверять типы, а ограничения — восстанавливать пропущенную информацию.

Инварианты вычисления

$$\Gamma \vdash M : \tau,\ M \to_\beta N \quad \Longrightarrow \quad \Gamma \vdash N : \tau$$

Прогресс и сохранение для выбранной стратегии **исключают тупики** у замкнутых типизируемых программ.

Завершение в чистом STLC

$$\Gamma \vdash M : \tau \quad \Longrightarrow \quad M \in \mathrm{SN}$$

Это **отдельная теорема**. При добавлении общей рекурсии она перестаёт выполняться.

Проверка понимания

Для каждого ответа приведите пример или укажите правило.

1. Почему $x\,x$ не типизируется, а $(\lambda x.x)(\lambda y.y)$ получает тип?
2. Чем главный тип терма Карри отличается от единственного типа фиксированного терма Чёрча?
3. Что нужно изменить при подстановке типа в $\lambda x : \alpha.x$?
4. Почему сохранение типа нельзя обратить по направлению β-редукции?
5. Может ли безопасная типизируемая программа вычисляться бесконечно?